

HANDS_ON_LabF5_Aggregation_Rollup

Lab F5: Aggregation / Rollup mit QueryRecord (file-only)

Apache NiFi 2.8.0, Single-User-Login (`admin`), UI unter <https://localhost:8443/nifi>
Fertiges Referenz-Lab auf der Canvas: “**Lab F5 - Aggregation (Rollup)**” Live
gebaut und verifiziert. **Keine Datenbank nötig.**

Lernziel: SQL-Aggregation über einen Datenstrom: aus einer CSV mit Einzelbestellungen eine **Zusammenfassung pro Region** erzeugen (Anzahl + Umsatz). Zeigt, dass NiFi mit QueryRecord GROUP-BY/SUM kann, ganz ohne Datenbank.

Flow:

```
GetFile —success—> QueryRecord —rollup—> PutFile
(CSV rein)          (GROUP BY region)      (Zusammenfassung raus)
```

Pfade

```
Input:  nifi-2.8.0/data/input/labF5
Output: nifi-2.8.0/data/output/labF5
Sample: nifi-2.8.0/data/samples/orders_rollup.csv
```

Eingangsdaten data/samples/orders_rollup.csv:

```
bestell_id,kundenname,region,betrag
1001,Mueller GmbH,Sued,149.90
1002,Schmidt AG,Nord,29.50
1003,Weber KG,West,210.00
1004,Fischer SE,Sued,75.00
1005,Klein OHG,Nord,49.90
1006,Gross AG,Sued,300.00
1007,Lang GmbH,West,12.50
1008,Kurz KG,Nord,88.00
```

Controller Services (im Process-Group-Kontext, beide aktivieren)

⚙ der Process Group → Tab **Controller Services** → +

CSVReader (liest die Eingangs-CSV): - Schema Access Strategy = `Infer Schema` - Treat First Line as Header = `true`

CSVRecordSetWriter (schreibt das Rollup): - Schema Access Strategy = `Inherit Record Schema`

Beide **enablen** (⚡).

`Infer Schema` lässt NiFi die Spalten/Typen selbst erkennen (`betrag` wird als Zahl erkannt, sonst könnte man nicht summieren). `Inherit Record Schema` übernimmt das Schema des QueryRecord-Ergebnisses.

Prozessoren

1. GetFile

- Input Directory = `.../data/input/labF5`
- File Filter = `.*\\.csv`
- Keep Source File = `false`
- Polling Interval = `2 sec` (Default 30s)

2. QueryRecord (das Herzstück)

- Record Reader = **CSVReader**
- Record Writer = **CSVRecordSetWriter**
- Dynamische Property (mit **+** anlegen): Name `rollup`, Wert:

```
SELECT region, COUNT(*) AS anzahl, SUM(betrag) AS umsatz FROM FLOWFILE GROUP BY region
```

- Relationships: **rollup** (= unsere Query) → PutFile; `original` und `failure` terminieren.

Der **Property-Name** wird zur **Relationship**. Mehrere Querys = mehrere Properties = mehrere Ausgänge. `FROM FLOWFILE` ist der feste Tabellename für den aktuellen Datenstrom.

3. PutFile

- Directory = `.../data/output/labF5`
 - Create Missing Directories = `true`, Conflict Resolution Strategy = `replace`
 - `success/failure` terminieren.
-

Verbinden

Von	Relationship	Nach
GetFile	success	QueryRecord
QueryRecord	rollup	PutFile
QueryRecord	original, failure	terminieren
PutFile	success, failure	terminieren

Starten und prüfen

```
cp data/samples/orders_rollup.csv data/input/labF5/  
sleep 6  
cat data/output/labF5/*
```

Verifiziertes Ergebnis (im Live-Test bestätigt)

```
region,anzahl,umsatz  
Sued,3,524.9  
Nord,3,167.4  
West,2,222.5
```

Aus 8 Einzelzeilen werden 3 aggregierte Zeilen: Anzahl Bestellungen und Umsatzsumme je Region.

Erweiterungsideen

- **Mehrere Auswertungen** = weitere QueryRecord-Properties (= weitere Ausgänge):
 - `grossauftraege` = `SELECT * FROM FLOWFILE WHERE betrag >= 100`
 - `pro_kunde` = `SELECT kundenname, SUM(betrag) AS umsatz FROM FLOWFILE GROUP BY kundenname`
- **Sortieren:** `... GROUP BY region ORDER BY umsatz DESC`
- **JSON-Output:** Writer auf `JsonRecordSetWriter` umstellen, selber Flow.
- **Filter + Aggregat kombiniert:** `... WHERE betrag > 20 GROUP BY region`

Reset

```
rm -f data/input/labF5/* data/output/labF5/*  
# Process Group stoppen
```

Häufige Stolpersteine

- `SUM(betrag)` **schlägt fehl / falsches Ergebnis** → `betrag` wurde als Text gelesen. CSVReader auf `Infer Schema` + saubere CSV (Punkt als Dezimaltrenner). Bei Semikolon-CSV: Value Separator `;`.
- **QueryRecord ohne Ausgang** → die SQL-Property wird erst durch Anlegen zur Relationship; danach verbinden/terminieren.
- **Reader/Writer nicht enabled** → Record-Prozessor bleibt invalid (△). Erst ⚡.
- **Leere Ausgabedatei** → `original` statt `rollup` verbunden; nur die `rollup`-Relationship trägt das Aggregat, `original` ist die unveränderte Eingabe.